

Table of Contents

Abstract.....	2
Introduction	3
This project builds a simple API system to solve these problems. The system allows users to:	3
Part A – API Design	3
1. GET /books	3
2. GET /books/search.....	3
3. POST /borrow	4
4. POST /return.....	4
5. GET /borrow/history/{user_id}	4
6. GET /overdue/{user_id}.....	4
7. GET /fines/{user_id}	5
System Flow	5
Part B – Implementation.....	6
The system uses Python with FastAPI.....	6
Key points:	6
Process example:	6
Screenshots:.....	7
Part C – Reflection	13
How APIs enhance library efficiency.....	13
SDG Alignment (SDG 4 – Quality Education):	14
Challenges encountered:.....	14
Future improvement:	14
Part D – GitHub Setup	14
Project Structure	14
Conclusion.....	16

Abstract

This project presents a web-based API for managing library operations. The system allows users to search for books, borrow them, and return them while tracking availability and user activity. The solution uses FastAPI with asynchronous functions to handle multiple requests at the same time. The goal is to improve efficiency, reduce manual work, and provide quick access to library resources.

Introduction

Many libraries still depend on manual processes. This slows down tasks like searching for books and tracking borrowed items.

This project builds a simple API system to solve these problems. The system allows users to:

- Search for books
- Borrow and return books
- Track borrowing activity
- Check overdue books and fines

The system focuses on speed, simplicity, and ease of use.

Part A – API Design

1. GET /books

Purpose: Retrieve all available books

Output example:

```
{  
  "1": {"title": "Learn Python", "available": true},  
  "2": {"title": "Data Science", "available": true}  
}
```

2. GET /books/search

Purpose: Search books by keyword

Input: query parameter

Example: ?query=python

3. POST /borrow

Purpose: Borrow a book

Input:

```
{  
  
  "user_id": 101,  
  
  "book_id": 1  
  
}
```

Output:

```
{  
  
  "message": "Book borrowed successfully"  
  
}
```

4. POST /return

Purpose: Return a book

Input:

```
{  
  
  "user_id": 101,  
  
  "book_id": 1  
  
}
```

5. GET /borrow/history/{user_id}

Purpose: View borrowing history

6. GET /overdue/{user_id}

Purpose: Check overdue books

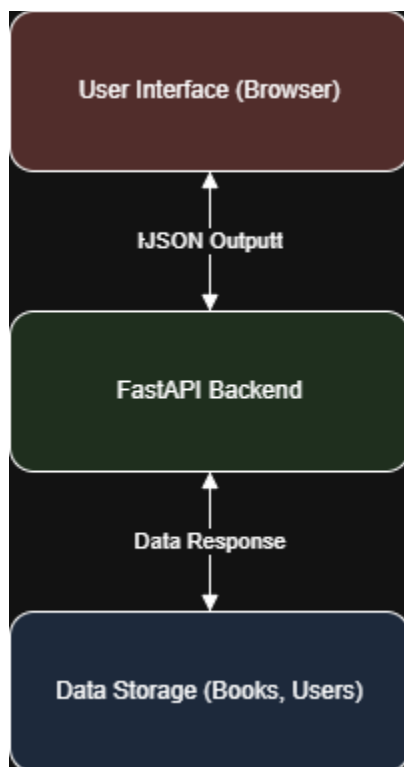
7. GET /fines/{user_id}

Purpose: Calculate fines

System Flow

- User sends request to API
- API processes request
- API interacts with data storage
- Data is returned to API
- API sends response back to user

Diagram



Part B – Implementation

The system uses Python with FastAPI.

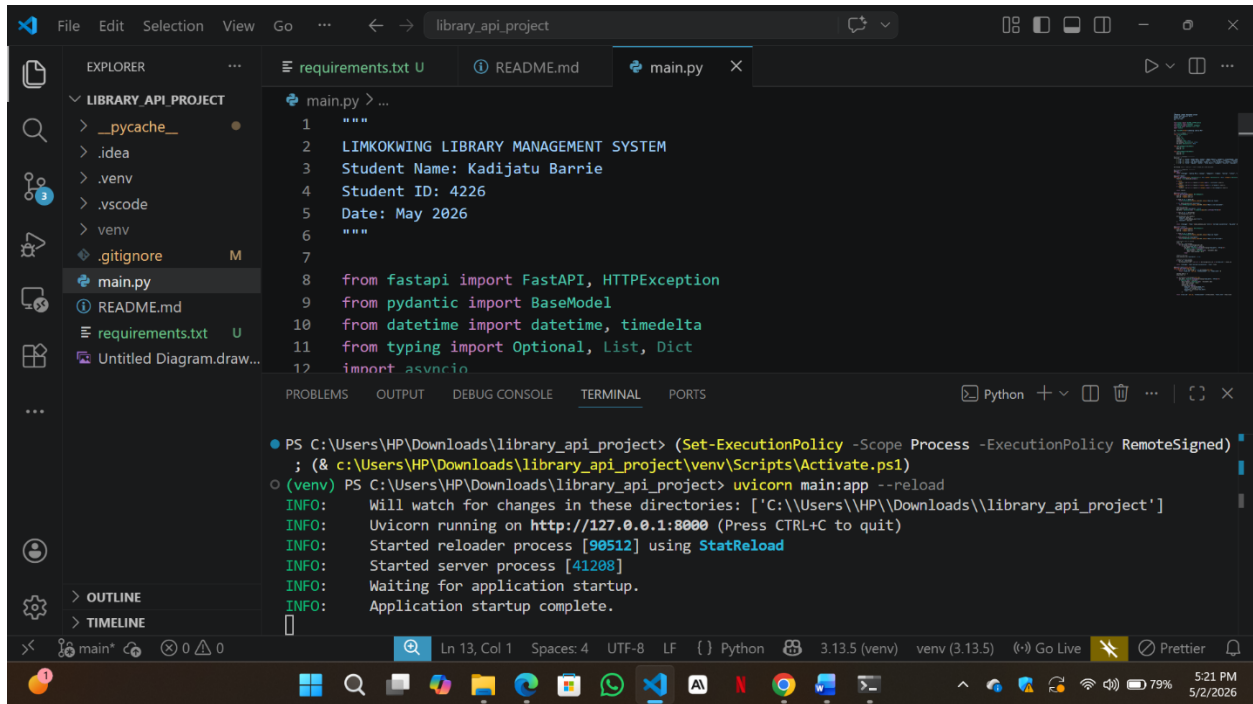
Key points:

- Async functions handle multiple users
- Data stored in dictionaries
- Simple logic for borrow and return

Process example:

- Check if user exists
- Check if book is available
- Update book status
- Store transaction
- Return response

Screenshots:

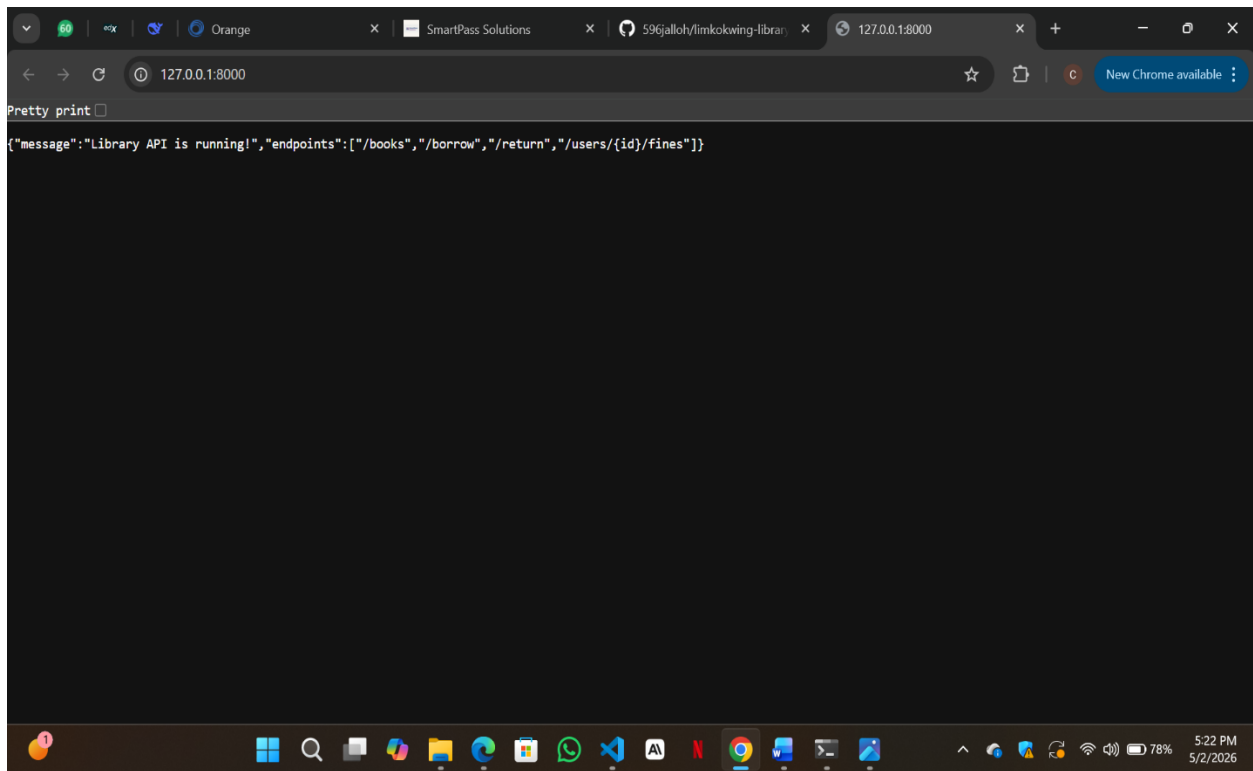


The screenshot shows the Visual Studio Code editor interface. The Explorer pane on the left displays the project structure for 'LIBRARY_API_PROJECT', including files like 'main.py', 'README.md', 'requirements.txt', and '.gitignore'. The main editor window shows the 'main.py' file with the following code:

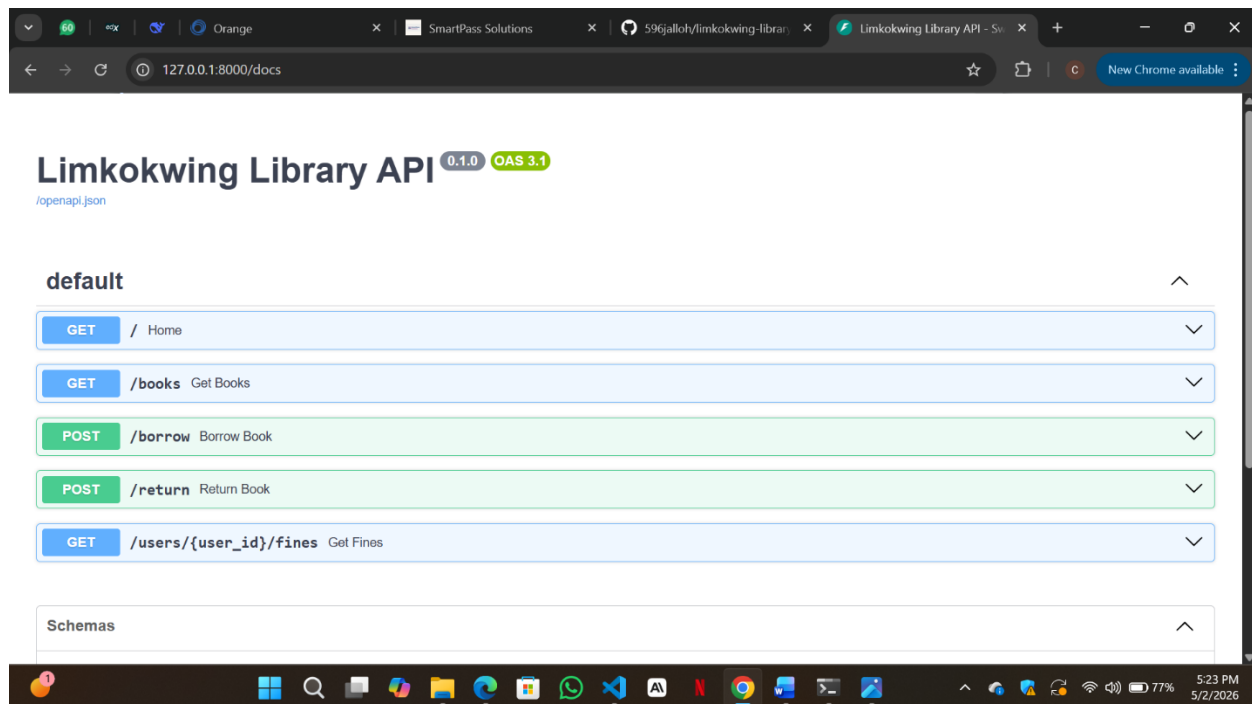
```
1 """
2 LIMKOKWING LIBRARY MANAGEMENT SYSTEM
3 Student Name: Kadijatu Barrie
4 Student ID: 4226
5 Date: May 2026
6 """
7
8 from fastapi import FastAPI, HTTPException
9 from pydantic import BaseModel
10 from datetime import datetime, timedelta
11 from typing import Optional, List, Dict
12 import asyncio
```

The terminal window at the bottom shows the command prompt output for running the application:

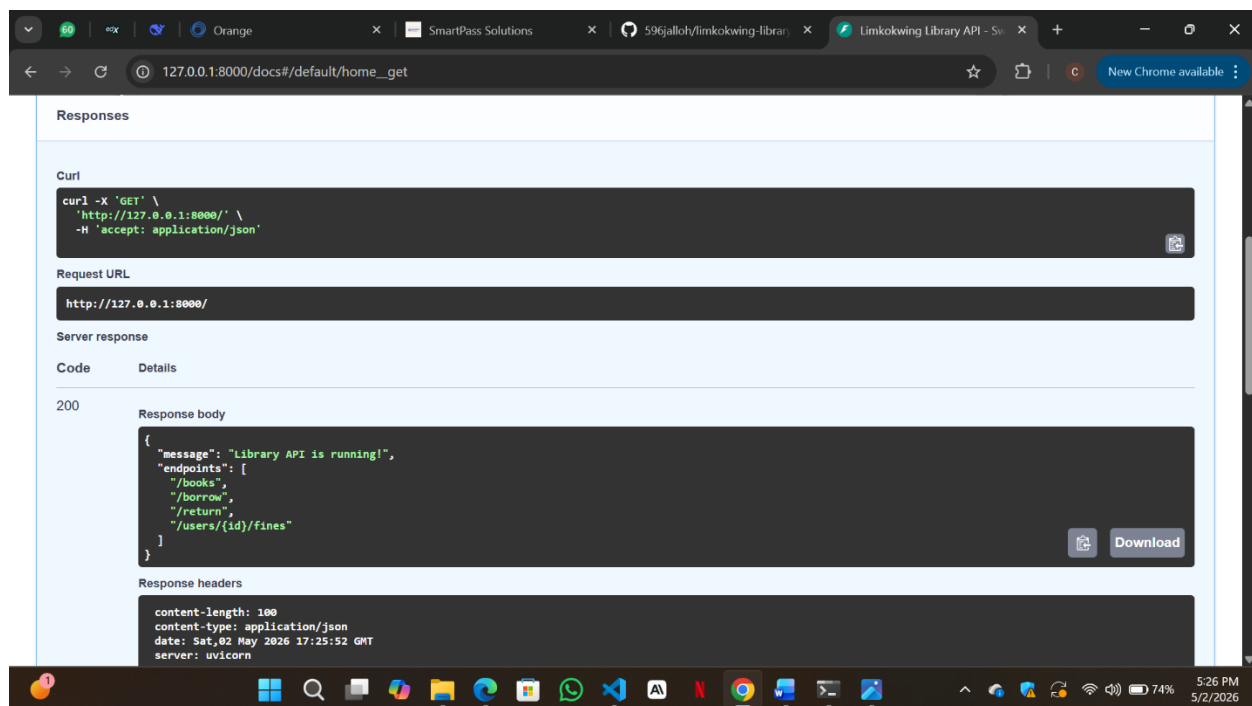
```
PS C:\Users\HP\Downloads\library_api_project> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned)
; (& c:\Users\HP\Downloads\library_api_project\venv\Scripts\Activate.ps1)
(venv) PS C:\Users\HP\Downloads\library_api_project> uvicorn main:app --reload
INFO: Will watch for changes in these directories: ['C:\Users\HP\Downloads\library_api_project']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [90512] using StatReload
INFO: Started server process [41208]
INFO: Waiting for application startup.
INFO: Application startup complete.
```



Open in the browser to do the test cases



Home



Get book

The screenshot shows a web browser window with the URL `127.0.0.1:8000/docs#/default/get_books_books_get`. The interface is for the 'GET /books' endpoint, titled 'Get Books'. It features a 'Parameters' section with a table for defining query parameters:

Name	Description
<code>title</code> string (string null) (query)	Clean Code
<code>author</code> string (string null) (query)	Robert Martin
<code>category</code> string (string null) (query)	Programming

At the bottom of the parameters section are two buttons: 'Execute' (in blue) and 'Clear' (in white). A 'Cancel' button is located in the top right corner of the parameters area.

Result

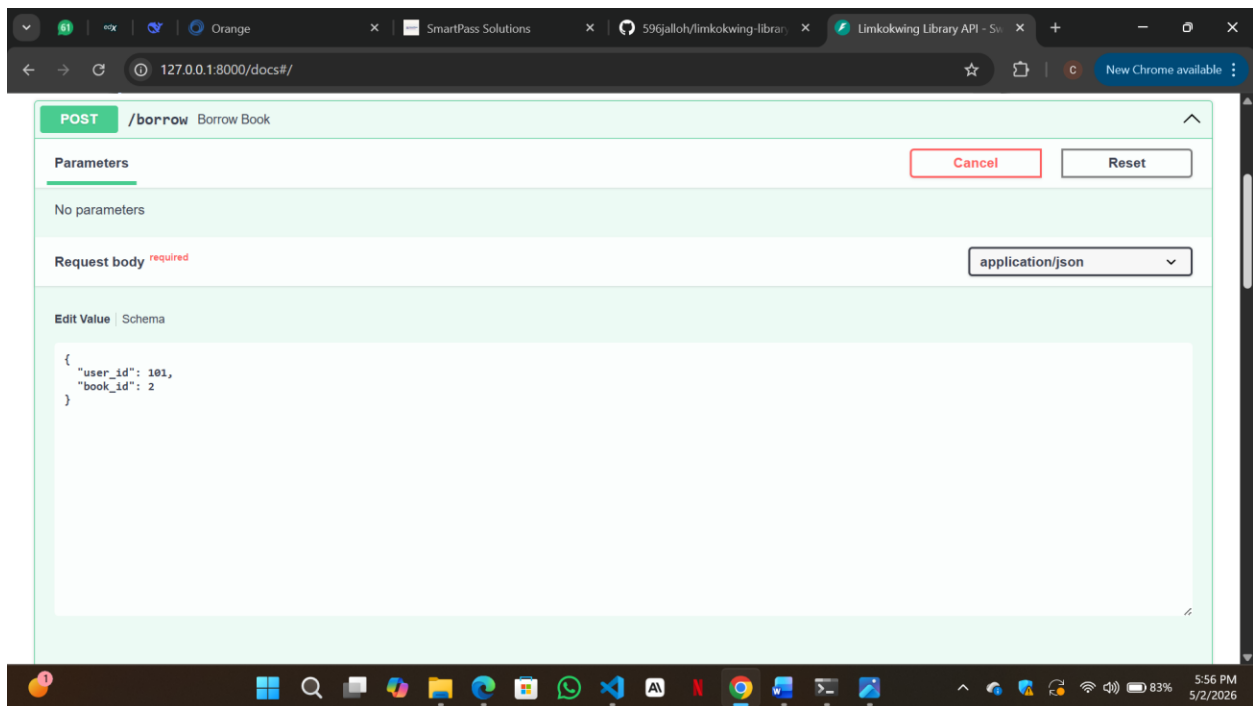
The screenshot shows the same web browser window, but now displaying the result of the executed GET request. The interface is divided into several sections:

- Curl:** A code block showing the curl command used:

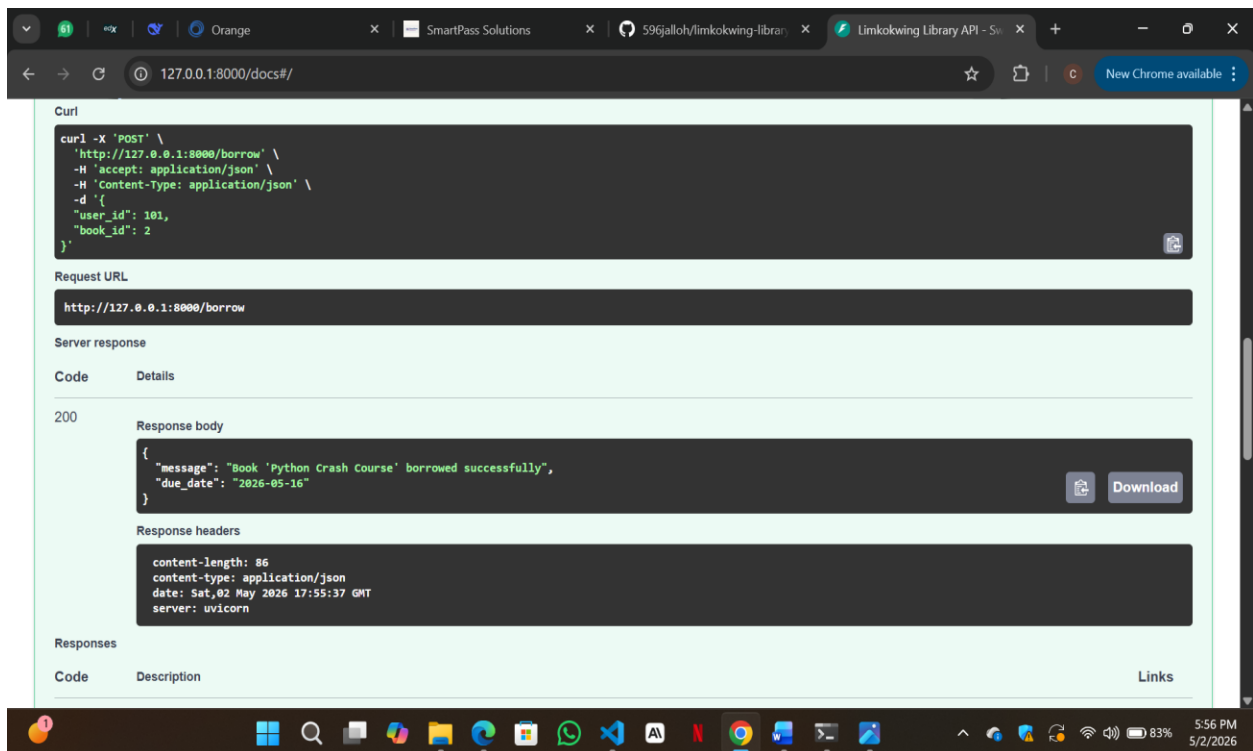
```
curl -X 'GET' \
'http://127.0.0.1:8000/books?title=Clean%20Code&author=Robert%20Martin&category=Programming' \
-H 'accept: application/json'
```
- Request URL:** A text box showing the full URL: `http://127.0.0.1:8000/books?title=Clean%20Code&author=Robert%20Martin&category=Programming`
- Server response:** A section with a table showing the response status and body.

Code	Details
200	Response body <pre>{ "id": 1, "title": "Clean Code", "author": "Robert Martin", "category": "Programming", "available": true }</pre> Response headers <pre>content-length: 98 content-type: application/json date: Sat, 02 May 2026 17:28:51 GMT server: uvicorn</pre>
- Responses:** A table at the bottom with columns for 'Code' and 'Description'.

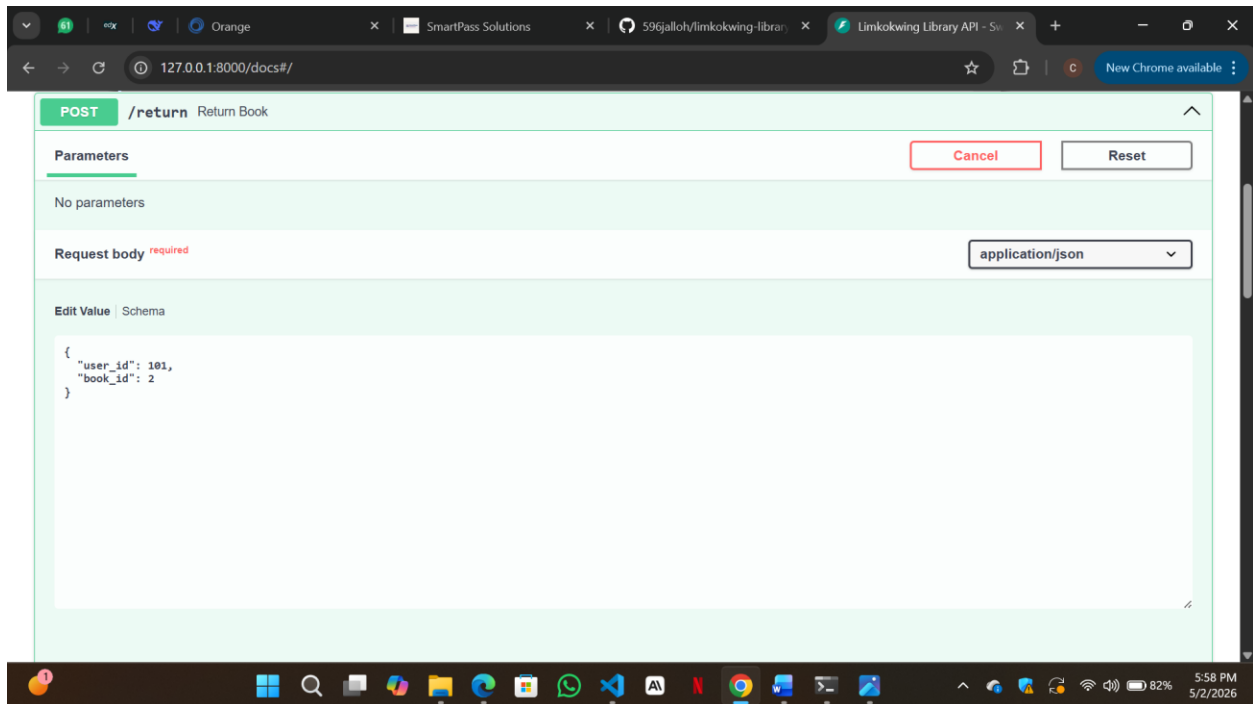
Post Borrow



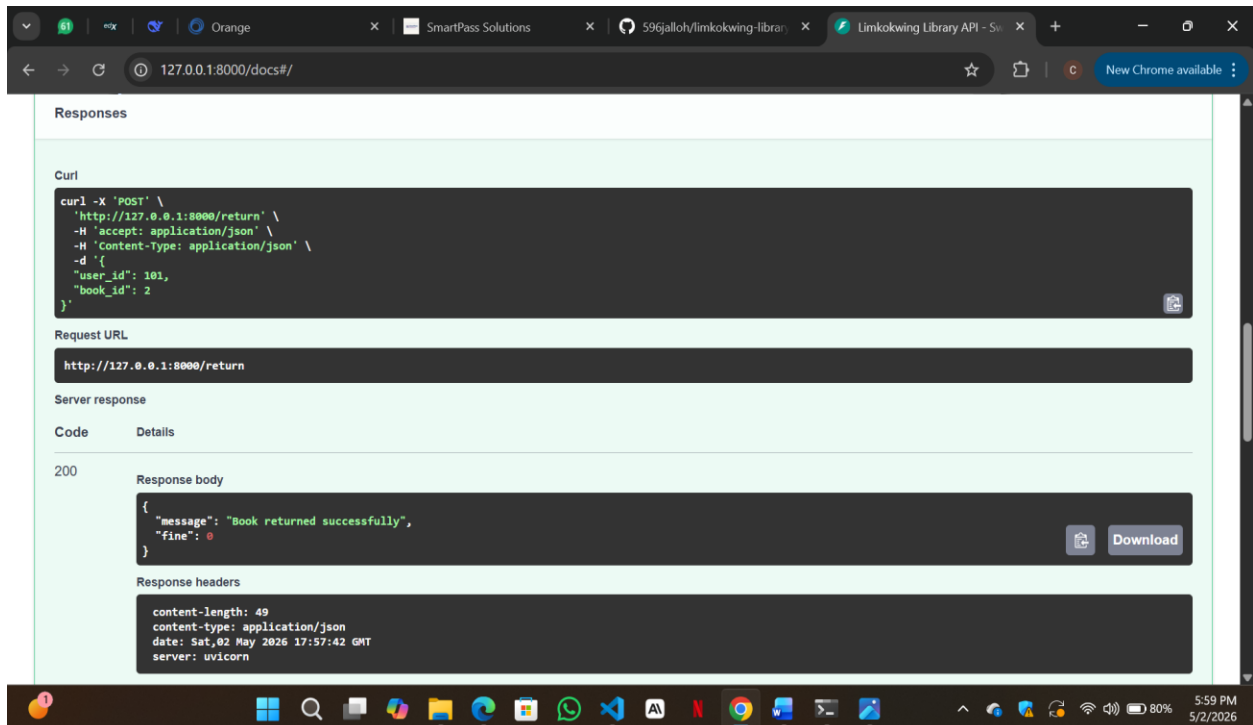
Result



Post return



Result



Get user fine

The screenshot shows the Limkokwing Library API interface in a web browser. The URL bar displays `127.0.0.1:8000/docs#/default/get_fines_users_user_id_fines_get`. The interface is for the `GET /users/{user_id}/fines` endpoint, titled "Get Fines".

Parameters:

Name	Description
<code>user_id</code> * required	integer (path)

The `user_id` parameter is set to `101`. Below the parameters, there are "Execute" and "Clear" buttons.

Responses:

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8000/users/101/fines' \
  -H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/users/101/fines
```

Result

The screenshot shows the Limkokwing Library API interface displaying the response for the `GET /users/{user_id}/fines` endpoint. The URL bar is the same as in the previous screenshot.

Responses:

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8000/users/101/fines' \
  -H 'accept: application/json'
```

Request URL

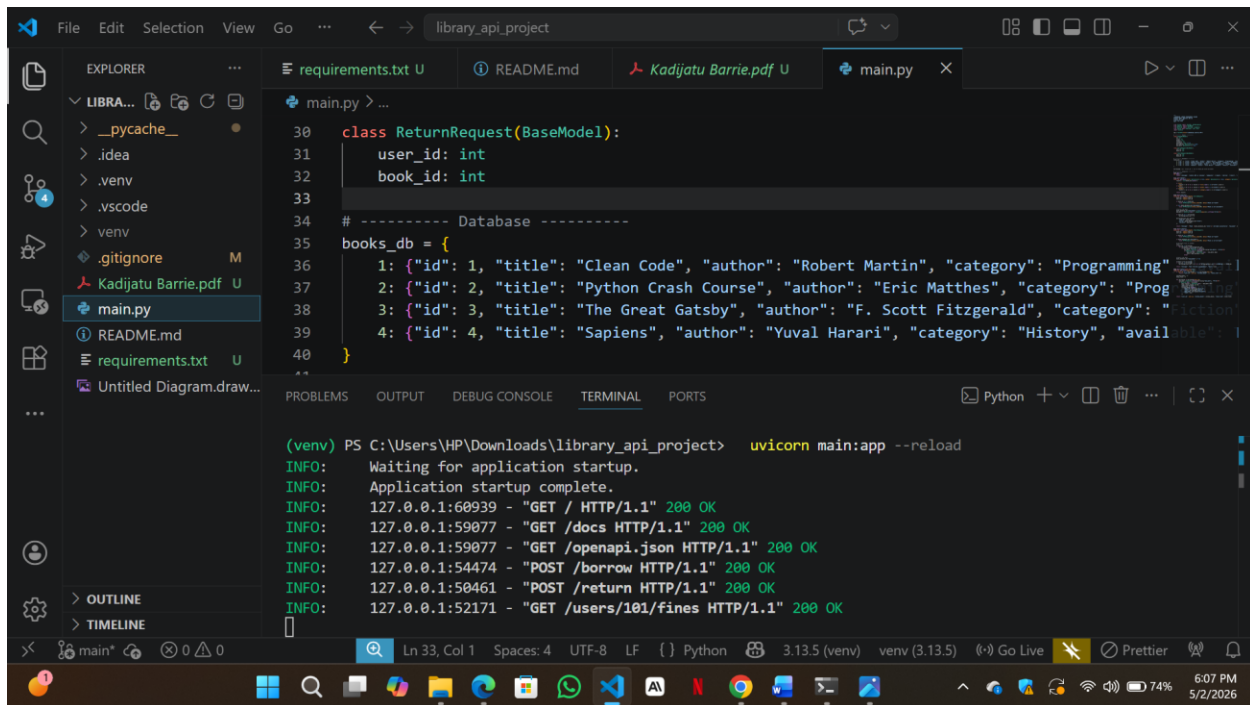
```
http://127.0.0.1:8000/users/101/fines
```

Server response

Code	Details
200	Response body <pre>{ "user_id": 101, "overdue_books": [], "total_fine": 0 }</pre> Response headers <pre>content-length: 49 content-type: application/json date: Sat, 02 May 2026 18:00:08 GMT server: uvicorn</pre>

At the bottom of the response details, there are "Download" and "Links" buttons.

After completing all test



```
class ReturnRequest(BaseModel):
    user_id: int
    book_id: int

# ----- Database -----
books_db = [
    {"id": 1, "title": "Clean Code", "author": "Robert Martin", "category": "Programming"},
    {"id": 2, "title": "Python Crash Course", "author": "Eric Matthes", "category": "Programming"},
    {"id": 3, "title": "The Great Gatsby", "author": "F. Scott Fitzgerald", "category": "Fiction"},
    {"id": 4, "title": "Sapiens", "author": "Yuval Harari", "category": "History", "available": True}
]
```

```
(venv) PS C:\Users\HP\Downloads\library_api_project> uvicorn main:app --reload
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: 127.0.0.1:60939 - "GET / HTTP/1.1" 200 OK
INFO: 127.0.0.1:59077 - "GET /docs HTTP/1.1" 200 OK
INFO: 127.0.0.1:59077 - "GET /openapi.json HTTP/1.1" 200 OK
INFO: 127.0.0.1:54474 - "POST /borrow HTTP/1.1" 200 OK
INFO: 127.0.0.1:50461 - "POST /return HTTP/1.1" 200 OK
INFO: 127.0.0.1:52171 - "GET /users/101/fines HTTP/1.1" 200 OK
```

Part C – Reflection

How APIs enhance library efficiency.

Application Programming Interfaces transform traditional library operations by creating a bridge between different software components. At Limkokwing, the lending API allows staff to process checkouts in under three seconds compared to manual logging which took several minutes. The /catalogue endpoint enables real-time inventory visibility, preventing staff from searching for books that are already borrowed. APIs also facilitate concurrent access – while one staff member processes a return, another can simultaneously check out different books without database conflicts. The penalty calculation endpoint automates what was previously a manual, error-prone process of checking due dates and calculating fees. Furthermore, APIs support mobile integration, allowing students to check their library status from smartphones. The structured JSON responses ensure consistency across different client applications (web, mobile, kiosk).

Overall, API-driven architecture reduces workload, minimizes errors, and improves service quality for all 15,000+ library members.

SDG Alignment (SDG 4 – Quality Education):

This library system directly supports SDG 4 (Ensure inclusive and equitable quality education and promote lifelong learning opportunities for all). By digitizing book lending with an efficient API, the library removes barriers to knowledge access. Students can quickly find, borrow, and return educational materials without bureaucratic delays. The automated fine system ensures fair resource turnover. This technological infrastructure promotes reading culture and supports academic success for all students regardless of background.

Challenges encountered:

1. **Race conditions** - When testing two members borrowing the last copy simultaneously, both transactions almost succeeded until implementing async locks
2. **Date handling** - Format inconsistencies between datetime objects and JSON serialization required careful conversion to ISO strings
3. **Member tracking** - Deciding whether to store borrowing history indefinitely or just active loans was a design trade-off

Future improvement:

Implement email notification system that automatically reminds members 2 days before books are due, reducing late fees and improving book availability and Add login authentication while use a real database.

Part D – GitHub Setup

Project Structure

library-api/

|

|— main.py

|— README.md

|— .gitignore

Description

The repository contains the full API system. It includes source code and setup instructions.

Conclusion

The API system improves how a library operates. It allows fast access to books and reduces errors. The system can grow by adding more features like authentication and database support.

References

- Fielding, R.T. (2000) Architectural Styles and the Design of Network-based Software Architectures. Available at: <https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm> (Accessed: 21 April 2026).
- IBM (2024) What is an API?. Available at: <https://www.ibm.com/topics/api> (Accessed: 21 April 2026).
- Python Software Foundation (2024) asyncio — Asynchronous I/O. Available at: <https://docs.python.org/3/library/asyncio.html> (Accessed: 21 April 2026).
- Red Hat (2023) What is a REST API?. Available at: <https://www.redhat.com/en/topics/api/what-is-a-rest-api> (Accessed: 21 April 2026).